

Module 2

Design & Verification Methodology (Part 2)

Learning objectives

- Basic TB vs UVM: From “wiggle pins and check” to “transaction → sequence → driver/monitor → scorebo...”
- UVM building blocks: Transaction (sequence item), sequence, driver, monitor, scoreboard, agent, env...
- Toolchain: UVM_HOME, include paths, Verilator flags (``-sv``, ``--timing``, ``--trace``), and ``make SIM=v...`

Prerequisites

- Module 1 completed (spec → RTL flow, spec_to_rtl example run).
- Verilator (5.036+)
- GNU Make and C++ compiler
- UVM: UVM_HOME set (must contain `src/uvm\_pkg.sv`) or vendored UVM in the repo (see uvm_smoke Makefi...

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Basic Testbench (Directed Te"]

T2["Evolution to UVM+SV Testbenc"]

T1 --> T2

T3["Toolchain (Verilator, UVM_HO"]

T2 --> T3

Understand basic testbench patterns (directed tests, pin wiggling), the evolution to a UVM+SV testbench (agents, sequences, drivers, monito...

Overview

- Module 2 builds on Module 1 (spec → RTL) by focusing on ****how we verify****:

Syllabus topics

1. Basic Testbench (Directed Tests, Pin Wiggling)

- Directed test: Drive specific inputs (e.g., reset, then a fixed sequence of data) and check expected...
- Pin wiggling: The testbench directly drives and samples DUT pins (clock, reset, data, enable) — no...

2. Evolution to UVM+SV Testbench

- Sequence: Produces a stream of transactions (e.g., a few directed values: 0x00, 0x01, 0x55, 0xAA, 0...
- Driver: Gets transactions from the sequencer and drives the DUT interface (e.g., set enable and d o...
- Monitor: Observes the DUT interface and turns pin activity into transactions, then sends them to th...
- Scoreboard: Compares “expected” (what we sent) vs “observed” (what the monitor saw). Reports matche...
- Agent: Groups driver, monitor, and sequencer for one interface. Env: Contains agent(s) and scoreboa...

3. Toolchain (Verilator, UVM_HOME, Make)

- Verilator: Compiles SystemVerilog (including UVM) + C++ into an executable. Key flags: ``-sv``, ``--ti...`
- UVM_HOME: Points to the UVM library root; the build uses ``$(UVM_HOME)/src/uvm_pkg.sv`` and ``+incdir+...`
- Make: One target to compile (Verilator + make in `obj_dir`), one to run (``./obj_dir/Vtest_uvm_smoke +...`

Command reference highlights

Environment checks

- `verilator --version`

Build and run uvm_smoke

- `cd module2/examples/uvm_smoke`

Module script (from repo root)

- `./scripts/module2.sh --check # Environment + UVM + example dirs`

Hands-on examples

Module 2 self-check

```
$ ./scripts/module2.sh --check
```

Terminal: module2_check

```
[[0;36m===== [[0m
[[0;36mModule 2: Demo commands (UVM smoke) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the uvm_smoke example (inside the example directory):
cd module2/examples/uvm_smoke
make SIM=verilator TEST=test_uvm_smoke

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd
# Then open in your waveform viewer, e.g.:
gtkwave *.vcd # if available

[[1;33m[INFO] [[0m See module2/EXAMPLES.md and module2/examples/uvm_smoke/README.md for more.
```

Example 1: UVM smoke test

- UVM phases and reporting (DRIVER, MONITOR, SCOREBOARD messages).
- Directed sequence: a few data values (0x00, 0x01, 0x55, 0xAA, 0xFF) driven and checked.
- Final scoreboard summary (matches/mismatches).

Demo: UVM smoke test

```
$ ./scripts/module2.sh --run
```

Terminal: ex01_uvm_smoke

From the repo root:

```
```bash
cd module2/examples/uvm_smoke
```

# Practice & assessment

# Exercises

- Run uvm\_smoke
- Traceability
- Change the sequence
- Optional: Waveforms

# Assessment checklist

- Can explain basic TB vs UVM (transaction, sequence, driver, monitor, scoreboard).
- Can describe the role of UVM\_HOME, Verilator flags, and Make in building/running a UVM test.
- Can run uvm\_smoke and interpret the output.
- Ready to move to Module 3 (UART protocol + RTL + basic testbench).

# Summary & next steps

- Understand basic testbench patterns (directed tests, pin wiggling), the evolution to a UVM+SV testb...
- Complete module2/CHECKLIST.md
- Run `./scripts/module2.sh --check`
- Next: UART